TECHNICAL PROJECT DOCUMENTATION
COMPUTER VISION – CCA 3

Image Restoration • Segmentation • Feature Extraction • Object Detection • Motion Estimation

Prepared from the submitted Jupyter Notebook: cv_tasks.ipynb

Submitted by

Pritam Sahu

PRN: 1272250539

Course instructor: Mr. Rahul Inchal

Date of submission: 04/11/2026

1. Executive Summary

This project implements a collection of classical and learning-based computer vision methods in Python. The work is organised into four practical task groups and implemented in a single Jupyter Notebook. It uses two photographs and one video as inputs. The experiments cover image degradation and restoration, segmentation, local feature extraction and matching, object detection, and video motion analysis.

The notebook uses OpenCV for image and video processing, NumPy for array operations, Matplotlib for visualisation, Pandas for tabular results, scikit-image for HOG and image-quality metrics, and Ultralytics for YOLO inference.

2. Project Objectives

- Introduce Gaussian and Salt-and-Pepper noise and investigate filtering methods.
- Measure restoration quality using Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM).
- Segment image regions using thresholding, morphological operations, Canny edges, and Watershed.
- Extract HOG and SIFT features, match local descriptors, and test rotation and scale changes.
- Compare Haar Cascade and pretrained YOLO object detection.
- Analyse video motion using frame differencing, MOG2 Gaussian Mixture Model background subtraction, and Lucas–Kanade optical flow.

3. Tools, Libraries and Input Data

Tool / Library	Role in this project
Python / Jupyter Notebook	Implementation environment; combines executable cells, explanations, and visual results.
OpenCV (cv2)	Image loading, filtering, thresholding, morphology, Canny, Watershed, SIFT, Haar Cascade, video handling, GMM, and optical flow.
NumPy	Numerical arrays, noise generation, clipping, kernels, and array manipulation.
Matplotlib	Displaying input images, intermediate results, masks, graphs, and comparisons.
Pandas	Organising image-quality measurements in a table.
scikit-image	HOG feature extraction and PSNR/SSIM metric functions.
Ultralytics YOLO	Pretrained YOLOv8 Nano object detection.

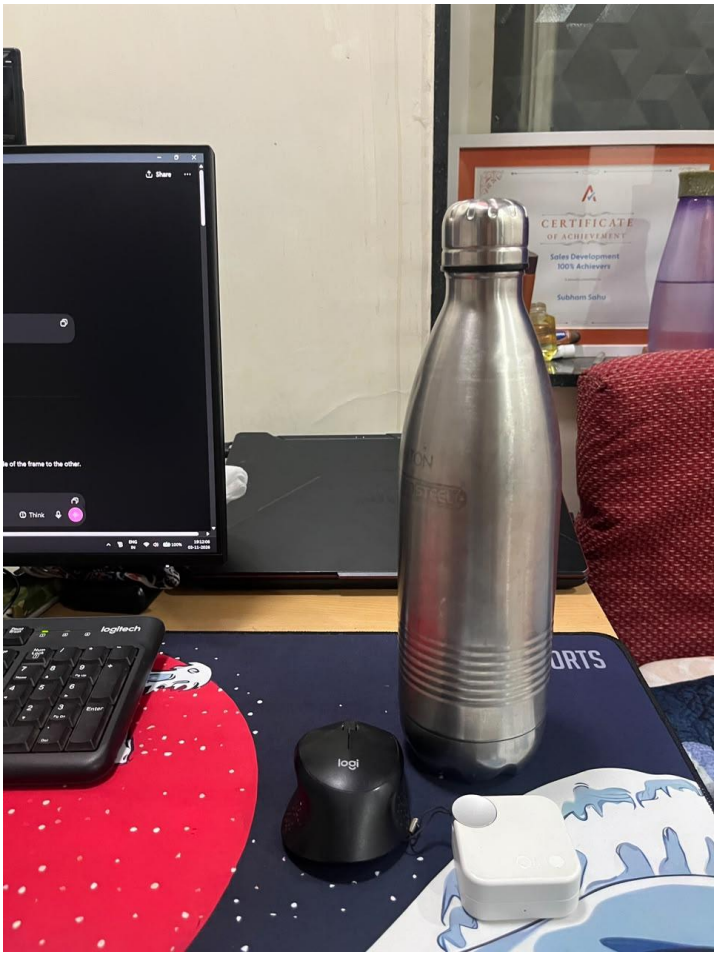
Input files

Input	Use
data/photo1.jpeg	Main photograph used for image restoration/segmentation and object detection.
data/photo2.jpeg	Second photograph used in HOG and SIFT feature extraction and matching.
data/video.mp4	Video input used for frame differencing, GMM background subtraction, and optical flow.

Input photograph 1



Input photograph 2



The supplied video is an input asset and is not embedded as a still image in this report.

4. System and Execution Workflow

- Place the notebook and the data folder in the same project directory.
- Confirm that the expected input files exist at data/photo1.jpeg, data/photo2.jpeg, and data/video.mp4.
- Install the required Python libraries in the same environment used by the notebook.
- Open cv_tasks.ipynb in Jupyter Notebook, JupyterLab, VS Code, or Google Colab.
- Run the notebook from the first cell in order. A fresh-kernel Run All test is recommended.
- Inspect the visual outputs and tables. The notebook saves selected outputs under outputs/.
- Save the executed notebook so its output cells are retained, then include the notebook and supporting assets in the submission repository.

5. Task 1 – Image Restoration and Quality Assessment

Aim: introduce two common image noise types, apply smoothing filters, and compare image quality against the original photograph.

5.1 Noise models

Gaussian noise is generated by sampling random values from a normal distribution and adding them to the image. The resulting pixel values are clipped to the valid 0–255 range and converted to 8-bit unsigned integers. Salt-and-Pepper noise is introduced by randomly selecting pixel coordinates and assigning white (salt) or black (pepper) values.

5.2 Restoration filters

Method	Notebook implementation	Purpose / expected behaviour
Gaussian filtering	cv2.GaussianBlur()	Smooths local intensity variations; can reduce Gaussian noise but may blur edges and fine detail.
Median filtering	cv2.medianBlur()	Replaces each pixel with a neighbourhood median; effective for isolated black/white impulse noise.

5.3 Quality evaluation

PSNR compares the mean squared error between a processed image and the reference image. Higher PSNR generally indicates lower pixel-level error. SSIM compares structural information such as luminance, contrast, and local structure; a value closer to 1 generally indicates greater similarity. The notebook calculates both metrics for the noisy and filtered images and presents them in a Pandas table and comparison graphs.

Interpretation should be based on the actual values displayed by the executed notebook. Filter performance depends on the image content, noise level, and selected kernel parameters.

6. Task 2 – Image Segmentation

Aim: divide an image into meaningful regions or identify boundaries and object shapes.

6.1 Binary thresholding

The image is converted to grayscale and thresholding is used to create a binary representation. Pixels on one side of the selected intensity threshold are assigned to the foreground class and the remaining pixels to the background class. The resulting mask is used as a basis for later segmentation operations.

6.2 Morphological operations

Operation	How it works	Typical effect
Erosion	Shrinks foreground regions using a structuring element.	Removes small protrusions and can separate narrow connections.
Dilation	Expands foreground regions.	Fills small gaps and connects nearby foreground pixels.
Opening	Erosion followed by dilation.	Can remove small foreground noise while retaining larger shapes.
Closing	Dilation followed by erosion.	Can fill small holes and close narrow gaps.

The notebook creates a 5×5 kernel for the morphology experiments.

6.3 Canny edge detection

Canny identifies edges using image gradients, non-maximum suppression, and hysteresis thresholding. The notebook applies Canny to a grayscale image and tests different threshold values to observe how the detected edge map changes.

6.4 Watershed segmentation

The Watershed workflow prepares a binary image, applies Otsu thresholding and morphological processing, identifies a sure-background region, and labels connected foreground components to form markers. These markers are supplied to the Watershed algorithm, which identifies region boundaries. The quality of the segmentation depends on the input image and the correctness of foreground/background markers.

7. Task 3 – Feature Extraction and Matching

Aim: represent image content using local gradient and keypoint features, then investigate how feature matching behaves between two photographs and transformed versions.

7.1 Histogram of Oriented Gradients (HOG)

HOG describes local image shape by computing gradient orientations within small spatial cells and normalising groups of cells. The notebook extracts HOG features from both photographs and displays the corresponding HOG visualisations. HOG is useful for representing local edge and shape patterns.

7.2 Scale-Invariant Feature Transform (SIFT)

SIFT detects distinctive local keypoints and computes a descriptor for each keypoint. The notebook uses cv2.SIFT_create() and detectAndCompute() on grayscale versions of the two photographs. Detected keypoints are drawn on the input images for visual inspection.

7.3 Descriptor matching

The notebook uses a Brute-Force matcher with the L2 norm and k-nearest-neighbour matching with k=2. Lowe's ratio test retains a match when the best descriptor distance is less than 0.75 times the second-best distance. The retained matches are drawn between the two photographs.

7.4 Rotation and resizing experiments

Transformation	Notebook setting	Evaluation
Rotation	Photo 2 is rotated by 45 degrees using cv2.getRotationMatrix2D() and cv2.warpAffine().	SIFT keypoints/descriptors are recalculated and good matches are counted.
Resizing	Photo 2 is resized by a factor of 1.5 in both dimensions using linear interpolation.	SIFT keypoints/descriptors are recalculated and good matches are counted.

The notebook prints the number of good matches for the original pair, the rotated pair, and the resized pair. These counts and the match visualisations should be used to discuss the observed effect. Matching depends on whether the two images depict the same object or overlapping scene, as well as image detail, viewpoint, blur, and lighting.

8. Task 4 – Object Detection and Motion Estimation

8.1 Haar Cascade face detection

The notebook loads OpenCV's haarcascade_frontalface_default.xml and converts the input photograph to grayscale. detectMultiScale() is called with scaleFactor=1.1, minNeighbors=5, and minSize=(30, 30). Green bounding boxes and the label 'Face' are drawn around detected regions. This specific cascade is trained for frontal faces and is not a general-purpose object detector.

8.2 YOLO object detection

The notebook loads the pretrained YOLOv8 Nano weights using YOLO('yolov8n.pt') and runs inference on the photograph. The annotated image is generated using results[0].plot(). The notebook also iterates through the predicted boxes to print the detected class name and confidence score. The first execution may need internet access to obtain the model weights. Detection categories are limited to the classes supported by the pretrained model.

8.3 Frame differencing

The video is opened using cv2.VideoCapture(). The notebook reads two consecutive frames, converts them to grayscale, and applies a 5×5 Gaussian blur. cv2.absdiff() calculates the absolute pixel difference. A binary threshold of 25 produces a motion mask. Contours are extracted from the mask, and bounding boxes are drawn around contours with area greater than 500 pixels.

8.4 GMM background subtraction (MOG2)

The notebook creates a MOG2 background subtractor with history=500, varThreshold=16, and detectShadows=True. Frames are processed sequentially, and a sample frame/mask is retained every 30 frames. For the selected sample mask, thresholding at 200 is used to exclude shadow values, followed by a 3×3 morphological opening to remove small foreground noise. Contours larger than 500 pixels are enclosed by bounding boxes.

8.5 Lucas–Kanade optical flow

Good feature points are selected in the first grayscale frame using cv2.goodFeaturesToTrack() with maxCorners=100, qualityLevel=0.3, minDistance=7, and blockSize=7. The next frame is tracked using cv2.calcOpticalFlowPyrLK(). The Lucas–Kanade configuration uses a 15×15 window, maxLevel=2, and termination criteria of 10 iterations or an epsilon of 0.03. Successfully tracked points are used to draw green motion trails and red circles.

8.6 Comparison of motion methods

Technique	What it measures	Strength / limitation
Frame differencing	Pixel changes between two consecutive frames.	Simple and direct; sensitive to noise, illumination changes, and camera motion.
GMM / MOG2	Foreground deviations from a background model learned over time.	Uses temporal background information; may require adaptation and can classify shadows or dynamic backgrounds.
Lucas–Kanade optical flow	Movement of selected feature points between frames.	Provides point-level motion vectors; depends on trackable features and successful correspondence.

9. Outputs Generated by the Notebook

The notebook saves selected experiment results in the outputs/ directory. The saved filenames visible in the notebook include:

- outputs/haar_detection.png
- outputs/yolo_detection.png
- outputs/frame_difference.png
- outputs/motion_mask.png
- outputs/motion_detection.png
- outputs/gmm_foreground_mask.png
- outputs/gmm_clean_mask.png
- outputs/gmm_motion_detection.png
- outputs/lucas_kanade_optical_flow.png
- outputs/motion_techniques_comparison.png
- outputs/sift_feature_matching.png
- outputs/rotated_photo2.png
- outputs/sift_rotation_matching.png
- outputs/resized_photo2.png
- outputs/sift_resize_matching.png

Other image restoration, segmentation, HOG, and keypoint results are displayed in notebook cells. Use the actual notebook save statements and executed outputs as the authoritative record of which files were generated.

10. Execution Instructions

10.1 Install dependencies

From a terminal in the project environment, install the libraries listed in requirements.txt:

```
python -m pip install -r requirements.txt
```

The project requires a compatible OpenCV installation with the functionality used by the notebook. Avoid installing multiple OpenCV wheel variants together in the same environment.

10.2 Prepare the project directory

```
CCA-3-Computer-Vision/
├── cv_tasks.ipynb
├── requirements.txt
├── README.md
├── data/
│   ├── photo1.jpeg
│   ├── photo2.jpeg
│   └── video.mp4
└── outputs/
```

10.3 Execute

- Start Jupyter Notebook/JupyterLab from the project directory, or open the notebook in Google Colab.
- Check that all three input files are present in the data folder and that the relative paths match the notebook.
- Run the notebook from the first cell to the last cell. Use a fresh kernel and Run All for a reproducibility check.
- Wait for YOLO model loading/inference and video processing to finish.
- Inspect every output, save the notebook, and verify that generated files exist in outputs/.

11. Technical Considerations and Limitations

- Results depend on the content and quality of the supplied photographs and video.
- PSNR and SSIM values should be taken from the actual notebook execution; they are not fixed project constants.
- Watershed performance depends on suitable markers and foreground/background separation.
- SIFT matching is meaningful when the input photographs contain shared visual content.
- Haar Cascade in this notebook is configured for frontal-face detection, while YOLO supports the categories encoded in its pretrained weights.
- Frame differencing uses only two consecutive frames in the implemented experiment; it does not build a persistent background model.
- MOG2 processes the video sequentially, while the displayed examples are sampled at 30-frame intervals.
- Lucas–Kanade optical flow tracks selected image features, not every pixel in the scene.
- Colab session storage is temporary; preserve original inputs and generated outputs outside the runtime.

12. Conclusion

The CCA 3 project demonstrates a broad workflow of practical computer vision methods. Image restoration experiments show how Gaussian and median filtering address different noise types and how PSNR/SSIM can be used for quality assessment. Segmentation experiments use intensity thresholds, morphology, edge detection, and marker-based Watershed. Feature extraction uses HOG and SIFT, with descriptor matching repeated after rotation and resizing. Finally, object detection is explored through Haar Cascade and YOLO, while video motion is studied using frame differencing, MOG2 background subtraction, and Lucas–Kanade optical flow. The notebook provides the executable implementation and visual evidence; conclusions about performance should be based on the actual outputs obtained from the supplied inputs.

13. Deliverables

- Executed Jupyter Notebook (cv_tasks.ipynb) containing code, explanations, and saved cell outputs.
- Input photographs and video in the data/ folder or an instructor-approved accessible location.
- Generated visual outputs in the outputs/ folder.
- requirements.txt for dependency installation.
- README.md explaining project setup and execution.
- This technical documentation document.